Diabetes Prediction System: Tools and Concepts Primer

Updated 29 October 2026. This primer teaches the audited Python baseline and labels future improvements separately. It does not imply that outstanding code defects have been repaired.

1. Python, notebooks and reproducibility

A notebook combines code, narrative and stored output. Stored output is a previous execution record, not proof that a fresh run succeeds. Reproducibility needs source, data, dependencies and a clear execution order.

The diabetes notebook has one substantive code cell. The audit executed that cell directly because Jupyter sockets were blocked locally. It reproduced the missing filename, the pandas 3.1 failure and the compatible-environment model result. Notebook-kernel execution remains unverified.

> Analogy: a photograph of a finished experiment does not prove that today's equipment and ingredients can reproduce it. A notebook's saved output is the photograph; a new execution is the experiment.

2. CSV, DataFrames and schema

CSV is text organised by delimiters. An Excel extension does not make a file an Excel workbook: the committed diabetes.xls is CSV text. The notebook expects diabetes.csv, causing the first startup failure.

pandas loads tabular data as a DataFrame with named columns. The schema should describe which inputs exist, their types, meanings, units and allowed values. This notebook uses eight positional inputs without an application-level schema validator.

Its current-pandas failure comes from supplying both columns and axis to drop. A compatible pandas 2.2 environment accepts the old call; removing the redundant axis argument is a proposed source repair.

3. Features, labels and a baseline

Features are model inputs; the label is the outcome being learned. Here eight features predict binary Outcome. The file has 500 negative and 268 positive labels, so ordinary accuracy needs a baseline.

Predicting class 0 for every test row would score 64.94% on the audited split. The SVM's 77.27% improves on that baseline but still misses 26 of 54 positives. A result should be judged against its objective, not against zero accuracy.

4. Missing values and coded values

No explicit NaN cells does not guarantee complete data. The file has zero-coded measurements in glucose, pressure, skin thickness, insulin and BMI; 376 rows are affected. The maintained benchmark notes identify this missing-data issue. Zero pregnancies should not be replaced indiscriminately.

The current notebook does not repair these values. A future design could mark designated measurements missing and fit an imputer on training data only. Whether that improves discrimination or recall must be measured.

> Analogy: a sensor's error code may look like an ordinary number. A model does not automatically know that the number means the sensor failed. Data semantics must be encoded before training.

5. Train/test splitting

A training partition fits the model; a held-out partition estimates behavior on unseen rows under the evaluation assumptions. Stratification helps preserve class proportions. A fixed seed gives a repeatable split, not proof that the split represents future data.

The notebook uses a stratified 80/20 split with seed 2: 614 train and 154 test rows. Test rows are not used to fit the scaler. Reusing the final holdout to repeatedly select models or thresholds would weaken its independence.

6. Standardisation and leakage

Standardisation subtracts a training mean and divides by a training standard deviation for each feature. StandardScaler stores those fitted statistics. Transform applies them without refitting.

The audit checked that the scaler saw 614 rows and its means matched training-only means. This diabetes notebook avoids scaling leakage. Standardisation still leaves zero-coded missing measurements as numerical values; it is not imputation.

> Analogy: learn the measuring ruler from training data, then keep it fixed during the exam. Letting test observations redesign that ruler is information leakage.

7. Linear SVM and regularisation

A linear SVM learns a separating hyperplane in feature space. Margin, support vectors and a regularisation trade-off determine the fitted boundary. Feature scale matters because it affects this geometry.

C controls the penalty/regularisation trade-off. The notebook chooses a linear SVC without a tuning search. It does not train the deep-learning models imported in the unrelated plant notebook for diabetes.

A decision score supports ranking; a predicted class is a thresholded decision; a calibrated probability is another quantity. The notebook does not establish calibrated disease risk.

8. Confusion matrices and metrics

Use class 1 as positive. The actual held-out counts are TN=91, FP=9, FN=26 and TP=28.

Metric	Formula	Here
Accuracy	(TP + TN) / all rows	77.27%
Precision	TP / (TP + FP)	75.68%
Recall	TP / (TP + FN)	51.85%

Metric	Formula	Here
Specificity	TN / (TN + FP)	91.00%
Balanced accuracy	(recall + specificity) / 2	71.43%

F1 combines precision and recall and is 0.6154 here. ROC AUC measures ranking across thresholds using scores; the audit obtained 0.7919. It is not the probability that an individual has diabetes and does not replace the confusion matrix at the chosen decision rule.

> Analogy: catching most mistakes and making few false alarms are different goals. Recall asks how many positives were found; precision asks how many positive alerts were correct. One percentage cannot answer both.

9. Cross-validation and uncertainty

Cross-validation repeats fitting on training folds and evaluation on held-out folds. Preprocessing must also be fitted inside each fold. The auditor used a scaling-plus-SVC Pipeline for stratified five-fold evaluation; the original notebook does not contain this evaluation.

Mean accuracy was 77.73% with 2.88 percentage-point fold standard deviation; mean positive recall was 57.05%. Ten split replays ranged from 68.83% to 79.87% accuracy. Do not pick the best split as the final advertised result.

Fold spread, a confidence interval and external validation answer different questions. The reported Wilson accuracy interval addresses sampling uncertainty under assumptions; it does not prove calibration, clinical suitability or performance on another population.

10. Input validation and persistence

Library checks reject wrong shape, non-numeric text, NaN and infinity. The tested sequence still classified all-zero and negative-valued inputs. Reversing the eight positions changed the example's class. Named input validation is a missing engineering layer.

A future saved bundle should include fitted preprocessing, feature names/order, classifier and environment/version metadata. Saving an SVM alone is insufficient if inference uses a different scaler. No saved diabetes bundle or serving path exists in the audited code.

11. Keep implemented work separate from proposals

Implemented: CSV loading, stratified split, training-only standardisation, linear SVC, accuracy reporting and a hard-coded example. Auditor-added: fuller metrics, independent checks, five-fold evaluation and ten split replays. Proposed: dependency packaging, validation, imputation, persistence and an application entrypoint.

The unrelated plant notebook requires missing Drive data and reuses training data as validation. It is not evidence of diabetes-model performance. No plant training result was reproduced.

12. Study path

Study order: tabular schema; split and scaling; SVM; confusion-matrix metrics; missing data; validation; uncertainty; reproducibility.

Sources: Reviewed code | StandardScaler | SVC | Pipeline | Benchmark missing-data notes